LAB NO. 05

INHERITANCE IN C++

(SINGLE, MULTILEVEL)

Lab Objectives

Following are the lab objectives:

1. Understand the concept of inheritance in Object-Oriented Programming.
2. Learn how to create base and derived classes in C++.
3. Implement single and multilevel inheritance in programs.
4. Apply inheritance to promote code reusability and better program structure.

Student ID 9864 **Student Name** Ali Raza

	Marks	Obtained Marks	Comments
Task 1	**15**		
Task 2	**15**		
Task 3	**15**		
Total Marks	**45**		

Lab Instructor

Inheritance in C++

The capability of a class to derive properties and characteristics from another class is called **Inheritance**. Inheritance is one of the most important features of Object-Oriented Programming in C++. In this article, we will learn about inheritance in C++, its modes and types along with the information about how it affects different properties of the class.

Syntax

```
class DerivedClass : mode_of_inheritance BaseClass {
    // Body of the Derived Class
};
```

where **mode of inheritance** controls the access level of the inherited members of the base class in the derived class. In C++, there are 3 modes of inheritance:

Mode	Description
Public Inheritance Mode	Public member of the base class will become public in the derived class and protected members of the base class will become protected in the derived class.
Protected Inheritance Mode	Both public and protected members of the base class will become protected in the derived class.
Private Inheritance Mode	Both public members and protected members of the base class will become private in the derived class. Private mode is the **default mode** that is applied when we don't specify any mode.

Access Base Class Members

Members of the base class can be accessed in the derived class by simply using their name.

```
class Base {
public:
    int n;
    void printN() {
        cout << n << endl;
    }
};

// Inheriting Base class publicly
class Derived : public Base {
    public:
    void func () {

        // Accessing Base class members
        n = 22;
    }
};
```

The public members of the **Base** class can be accessed through the objects of the **Derived** class if the **Base** class is inherited publicly as in the above example.

```
#include <iostream>
using namespace std;

class Base {
public:
    int n;
    void printN() {
        cout << n << endl;
    }
};

// Inheriting Base class publicly
class Derived : public Base {
    public:
    void func () {

        // Accessing Base class members
        n = 22;
    }
};
```

```
int main() {

    // Creating objects of derived
    Derived d;

    // Accessing Derived class member
    d.func();

    // Accessing Base class member
    d.printN();
    return 0;
}
```

Output

```
22
```

The private members in the base class cannot be directly accessed in the derived class, while protected and public members can be directly accessed. To access or update the private members of the base class in derived class, we have to use the corresponding getter and setter functions of the base class or declare the derived class as friend class.

Types Of Inheritance in C++

The inheritance can be classified on the basis of the relationship between the derived class and the base class. In C++, we have **5 types of inheritances:**

- Single inheritance
- Multilevel inheritance
- Multiple inheritance
- Hierarchical inheritance
- Hybrid inheritance

1. Single Inheritance

In single inheritance, a class is allowed to inherit from only one class. i.e. one base class is inherited by one derived class only.



Single Inheritance

Example:

```cpp
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

// Sub class derived from a single base classes
class Car : public Vehicle {
public:
    Car() {
        cout << "This Vehicle is Car"<< endl;
    }
};

int main() {

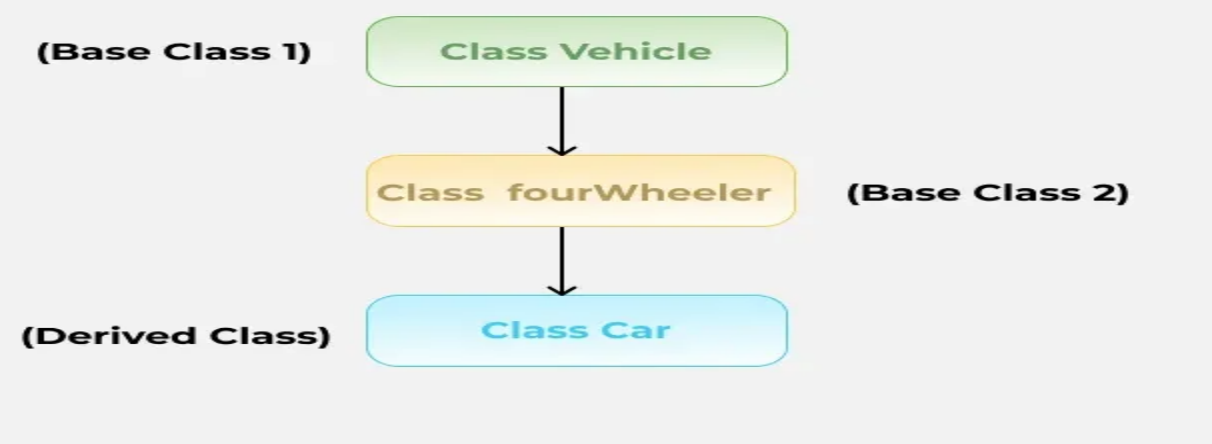
    // Creating object of sub class will
    // invoke the constructor of base classes
    Car obj;
    return 0;
}
```

Output

```
This is a Vehicle
This Vehicle is Car
```

3. Multilevel Inheritance

In multilevel inheritance, a derived class is created from another derived class and that derived class can be derived from a base class or any other derived class. There can be any number of levels. For example, a vehicle can be a four-wheeler, and a four-wheeler vehicle can be a car.



Multilevel Inheritance

Example:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

class fourWheeler : public Vehicle {
public:
    fourWheeler() {
        cout << "4 Wheeler Vehicles"<< endl;
    }
};

class Car : public fourWheeler {
public:
    Car() {
        cout << "This 4 Wheeler Vehical is a Car";
    }
};
```

```
int main() {

    // Creating object of sub class will
    // invoke the constructor of base classes.
    Car obj;
    return 0;
}
```

Output

```
This is a Vehicle
4 Wheeler Vehicles
This 4 Wheeler Vehical is a Car
```

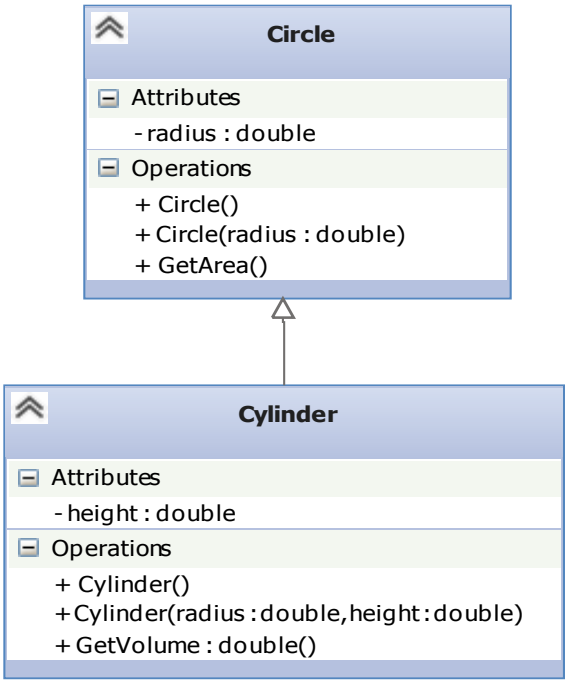
LAB TASKS

Task 1: Cylinder Inherits Circle (15 marks)

See the following class diagram of **circle** & **cylinder** and implement inheritance relationship of classes in python.

You need to implement following items for each class;

1. Constructors
2. Setters
3. Getters
4. Member Methods



(Past your code here.)

Program:

```
#include <iostream>

using namespace std;
```

```
class circle {
private:
    double radius;
    double PI = 3.14;

public:
    circle(double r) {
        radius = r;
    }

    double area() {
        return PI * radius * radius;
    }

    double circumference() {
        return 2 * PI * radius;
    }

    double getRadius() {
        return radius;
    }

    void display() {
        cout << "Radius of circle: " << radius << endl;
        cout << "Area of circle: " << area() << endl;
        cout << "Circumference of circle: " << circumference() << endl;
    }
};

class cylinder : public circle {
private:
    double height;
```

```
public:
    cylinder(double r = 14.0, double h = 8.0) : circle(r) {
        height = h;
    }

    void setHeight(double h) {
        height = h;
    }

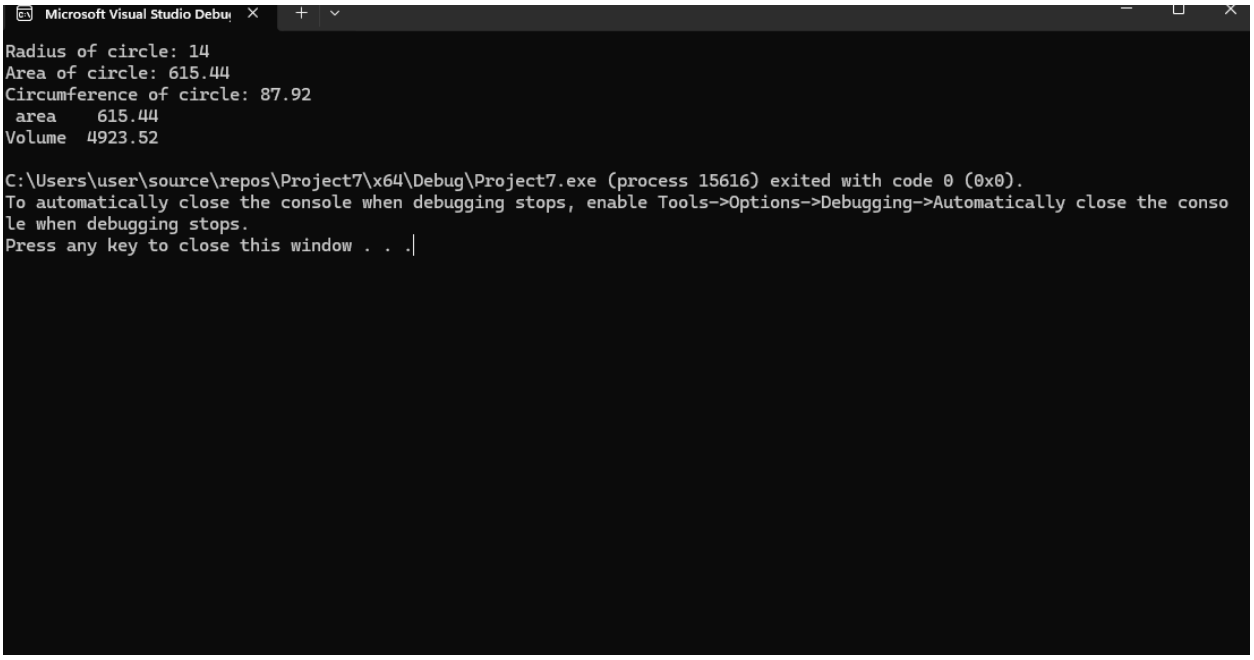
    double getHeight() {
        return height;
    }

    double volume() {
        return area() * height;
    }

    void display() {
        circle::display();
        cout << " area    " << area()<< endl;
        cout << "Volume  " << volume() << endl;
    }
};

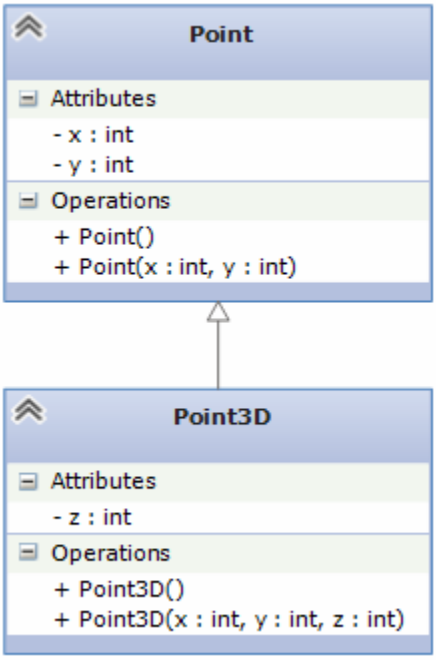
int main() {
    cylinder c(14.0, 8.0);
    c.display();
    return 0;
}
```

Output:


Microsoft Visual Studio Debu
Radius of circle: 14
Area of circle: 615.44
Circumference of circle: 87.92
area 615.44
Volume 4923.52
C:\Users\user\source\repos\Project7\x64\Debug\Project7.exe (process 15616) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .

Task2 : Point & Point3D **(15 marks)**

See the following class diagram of **point** & **point3d** and implement inheritance relationship of classes in python.




```
#include <iostream>
using namespace std;


class Student {
public:
    int rollNumber;
    int marks;

    Student() {
        rollNumber = 0;
        marks = 0;
```

```
    }

    Student(int r, int m) {
        rollNumber = r;
        marks = m;
    }
};


class StudentRecord : public Student {
public:
    int semester;

    StudentRecord() {
        semester = 0;
    }

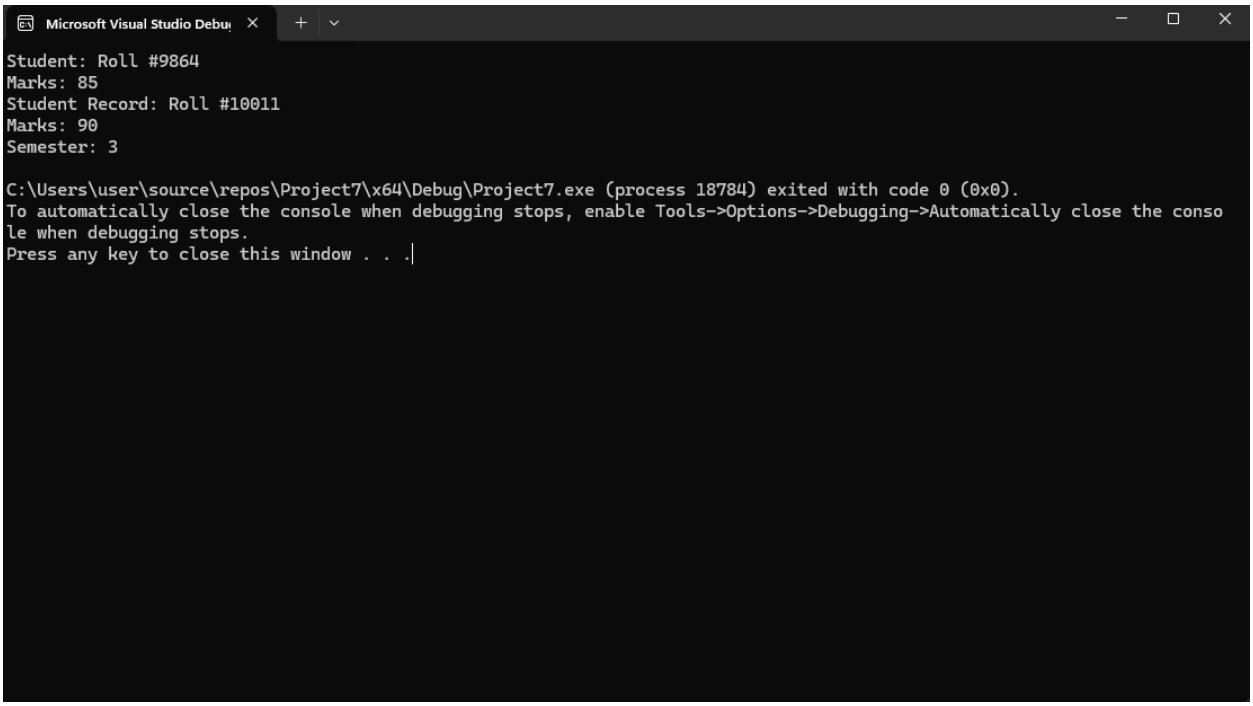
    StudentRecord(int r, int m, int s) : Student(r, m) {
        semester = s;
    }
};

int main() {
    Student s1(9864, 85);
    StudentRecord s2(10011, 90, 3);

    cout <<"Student: Roll #" << s1.rollNumber << endl;
    cout<< "Marks: " << s1.marks << endl;
    cout << "Student Record: Roll #" << s2.rollNumber << endl;
    cout << "Marks: " << s2.marks << endl;
    cout<<"Semester: " << s2.semester << endl;

        return 0;
```

Output:



}

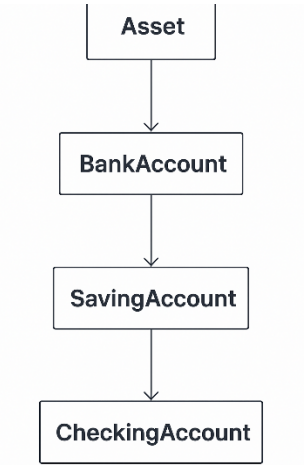
Task 3: Multi-level Inheritance (15 marks)

See the following class diagram and implement multi-level inheritance relationship of classes in C++.

You need to implement following items for each class;

1. Constructors
2. Setters
3. Getters
4. Member Methods

Demonstrate working of your classes.



(Past your code here.)

```
#include <iostream>
#include <string>
using namespace std;

class Asset {
public:
    string ownerName;
    int ownerAge;

    Asset() {}
    Asset(string name, int age) {
```

```
        ownerName = name;
        ownerAge = age;
    }

    void setOwner(string name, int age) {
        ownerName = name;
        ownerAge = age;
    }

    string getOwnerName() {
        return ownerName;
    }

    int getOwnerAge() {
        return ownerAge;
    }

    void showAssetInfo() {
        cout << "Owner: " << ownerName << ", Age: " << ownerAge << endl;
    }
};

class BankAccount : public Asset {
public:
    string accountNumber;
    double balance;

    BankAccount() {}
    BankAccount(string name, int age, string accNum, double bal)
        : Asset(name, age) {
        accountNumber = accNum;
        balance = bal;
    }
```

```
    void setAccount(string accNum, double bal) {
        accountNumber = accNum;
        balance = bal;
    }

    string getAccountNumber() {
        return accountNumber;
    }

    double getBalance() {
        return balance;
    }

    void showBankInfo() {
        showAssetInfo();
        cout << "Account Number: " << accountNumber << ", Balance: " << balance << endl;
    }
};

class SavingAccount : public BankAccount {
public:
    double interestRate;

    SavingAccount() {}
    SavingAccount(string name, int age, string accNum, double bal, double rate)
        : BankAccount(name, age, accNum, bal) {
        interestRate = rate;
    }

    void setSavingDetails(double rate) {
        interestRate = rate;
    }
```

```
    double getInterestRate() {
        return interestRate;
    }

    void showSavingInfo() {
        showBankInfo();
        cout << "Interest Rate: " << interestRate << "%" << endl;
    }
};

class CheckingAccount : public SavingAccount {
public:
    double transactionFee;

    CheckingAccount() {}
    CheckingAccount(string name, int age, string accNum, double bal, double rate, double fee)
        : SavingAccount(name, age, accNum, bal, rate) {
        transactionFee = fee;
    }

    void setCheckingDetails(double fee) {
        transactionFee = fee;
    }

    double getTransactionFee() {
        return transactionFee;
    }

    void showCheckingInfo() {
        showSavingInfo();
        cout << "Transaction Fee: " << transactionFee << endl;
    }
```

```
};

int main() {
    CheckingAccount acc("Sajal Tariq", 9, "ACC123", 50000.0, 3.5, 50.0);
    acc.showCheckingInfo();

    return 0;
}
```

Output:

